Module 4: Operator overloading

GOPAS

Overview

- Why do I need to overload operators
- Operators that can be overloaded
- How to overload operators
  - overloading +
  - overloading ++
  - overloading == and !=
  - overloading conversion operators

Operators and Methods

- Using methods
  - Reduces clarity
  - Increases risk of errors, both syntactic and semantic

```
myIntVar1 = Number.Sum(number1,
                       Number.Sum(Number.Sum(number2,
                                             number3), 4));
```

- Using operators makes expressions clear

```
myIntVar1 = number1 + number2 + number3 + 4;
```

Conventions of overloading operators

- Operator function **must be a member** function of the containing type.
- The operator function **must be static**.
- Function must have the keyword **operator**
- The **arguments of the function are the operands**.
- The **return value of the function is the result of the operation**.

```
public static Time operator+(Time t1, Time t2)
{
}
```

Overloading operator +

- Operator syntax

```
public static Time operator+(Time t1, Time t2)
{
        int newHours = t1.hours + t2.hours;
        int newMinutes = t1.minutes + t2.minutes;
        return new Time(newHours, newMinutes);
}
```

Note:

- **+=** *operator is evaluated using overload of operator* **+**

Overloading Operators Multiple Times

- The same operator can be overloaded multiple times

```
public static Time operator+(Time t1, int hours)
public static Time operator+(int hours, Time t1)

public static Time operator+(Time t1, float hours)
public static Time operator+(float hours, Time t1)
```

- Not commutative by default
  - For example String "addition" is not commutative!!!

Overloading operator ++

- operator ++ is unary operator

```
public static Date operator ++(Date date)
{
}
```

Note:

- *May not modify its operand directly but it should make a new instance*

```
public static Date operator ++(Date date)
{
    return new Date(date.Day + 1, date.Month, date.Year);
}
```

Overloading operator == and !=

- By default, the operator == tests **for reference equality** for reference types
- overloading operator == to compare value equality instead of reference equality can be useful **for immutable types**
- Type that overloads operator == should also overload operator !=

```
public static bool operator ==(Time t1, Time t2)
{
    return t1.Equals(t2);
}
public static bool operator !=(Time t1, Time t2)
{
    return !t1.Equals(t2);
}
```

Note:

- *Operator == implementations should not throw exceptions.*

Overloading Conversion Operators

```
float f = 12354;
Time t = (Time)f;
```

- Overloaded conversion operators

```
public static explicit operator Time(float hours)
{ ... }
```

```
public static implicit operator string(Time t1)
{ ... }
```

Note:

- *If a class defines a string conversion operator, the class should override also ToString()*
- *The "**as**" operator never uses user-defined conversions*

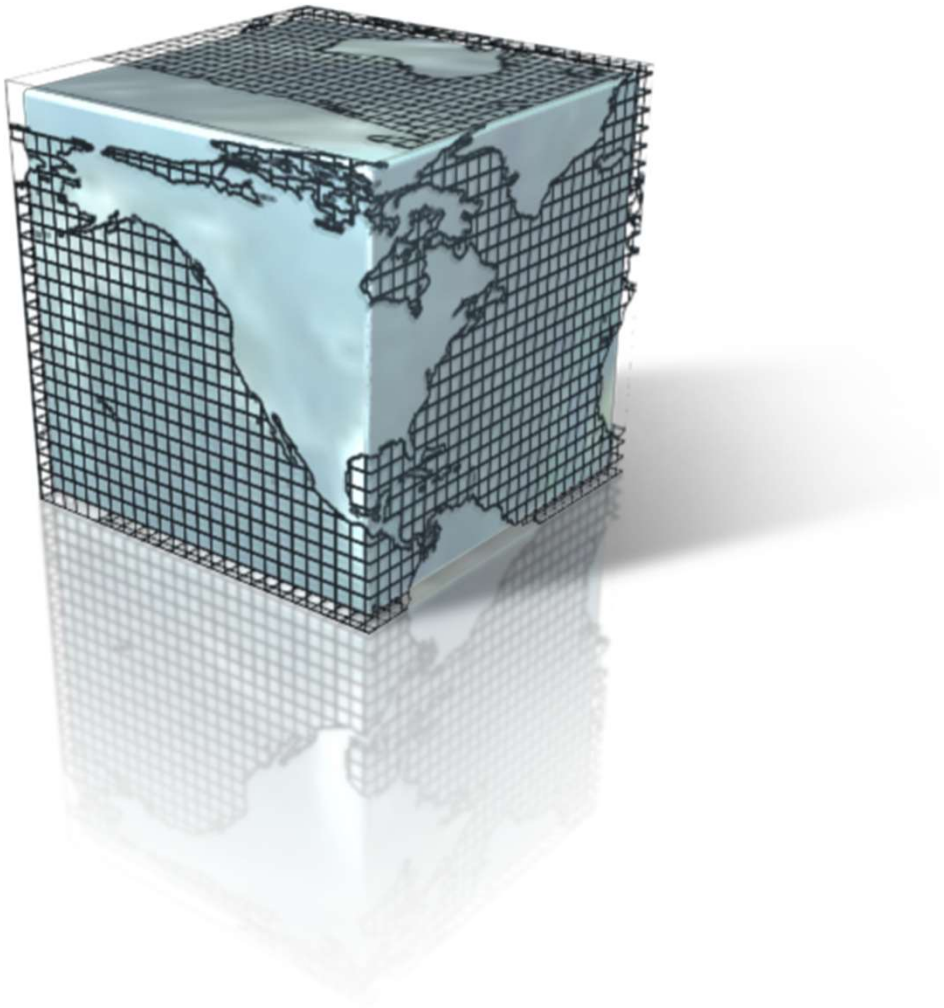
Overloading Operators

- All unary and binary operators have predefined implementations
- You can redefine or overload most of the built-in operators available in C#
- User-defined operator implementations always take precedence over predefined operator implementations

Note:

- *Most operators in c# can be overloaded (==, !=, <, >, <=, >=, +, -, !, ~, ++, --, *, /, %, &, |, ^)*
- *These operators can't be overloaded (=, ., ?:, ->, new, is, sizeof, typeof)*

Lab 04.1 - Operator overloading

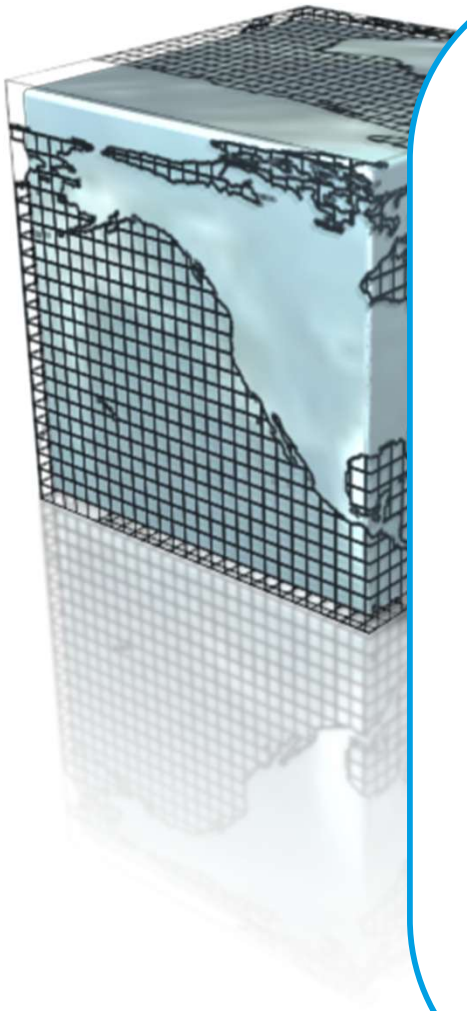


Example:

1. Intro
2. Overloading operator+
3. Overloading operator++
4. Overloading operator== and !=
5. Overloading cast operator ()

GOPAS

Exercise - Overloading Operator ==



```
class Program
{
        static void Main(string[] args)
        {
            Point locationA = new Point(50, 50);
            Point locationB = new Point(50, 50);

            if (locationA == locationB)
                Console.WriteLine("Their locations are the same");
            else
                Console.WriteLine("Their locations  are different");
        }
}

class Point
{
        public int x;
        public int y;

        public Point(int x, int y)
        {
            this.x = x;
            this.y = y;
        }
}
```

GOPAS

Quiz: Spot the Bugs

```
public bool operator != (Time t1, Time t2)
{ ... }
```

```
public static operator float(Time t1) { ... }
```

```
public static Time operator += (Time t1, Time t2)
{ ... }
```

```
public static bool Equals(Object obj) { ... }
```

```
public static int operator implicit(Time t1)
{ ...}
```

Review

- Why do I need to overload operators
- Operators that can be overloaded
- How to overload operators
  - overloading +
  - overloading ++
  - overloading == and !=
  - overloading conversion operators